

ECC Pipeline — Technical Reference

Stack: Python 3.10+, asyncio, aiohttp, aiosqlite, aiodns, SQLite WAL

Table of Contents

1. Architecture
 2. Component Breakdown
 3. Record State Machine
 4. Output Schema
 5. Cost Model
 6. Performance
 7. Live Run — 500 Records
-

1. Architecture

```
Input JSONL
|
v
ProducerWorker (asyncio)
  chunk_size=100 records processed concurrently
  1. DNS probe -> aiodns MX on derived domain stems
  2. Serper    -> Google search if DNS missed
  3. Pattern gen -> email candidates from domain + name
  Writes record_state=DISCOVERED to SQLite
  Signals consumer via FIFO named pipe after each batch
  |
  | Named pipe (FIFO) -- <1ms wake
  v
pipeline/bbops.py (subprocess)
  Reads all DISCOVERED records
  Submits candidate emails to bbops.io SMTP batch API
  Polls until jobs complete
  Updates: VALIDATED (bbops_valid) or VALIDATION_FAILED
  |
  v
ConsumerWorker (asyncio)
  Atomically claims DISCOVERED rows (UPDATE RETURNING)
  For each record:
    1. MS probe -> GetCredentialType (free, MS 365 only)
    2. Zuhal    -> paid SMTP validation (last resort)
  Updates pattern_stats after each verdict
  |
  v
_write_outputs() [shutdown only]
-> pipeline.db      (always on disk)
-> results.json     (run summary)
-> valid_emails.csv (VALIDATED records only)
```

Key design decisions

Decision	Rationale
SQLite + WAL mode	Single-server; WAL enables concurrent producer writes and consumer reads without locking
isolation_level=None	Disables Python's implicit BEGIN — allows explicit transaction control in insert_records_batch
Atomic UPDATE RETURNING claim	Eliminates SELECT+UPDATE race condition that existed in V1/V2
Shared aiodns.DNSResolver	One c-ares context per run — negative-TTL caching; no per-record setup overhead
Named pipe IPC	Consumer wakes in under 1ms vs 5s fixed poll interval
enrichment_cache 30-day TTL	Serper responses cached by business name, agent name, state, provider — re-runs cost \$0 on Serper
pattern_stats per MX provider	Templates ranked by historical success rate — improves first-candidate hit rate across runs
aiobreaker circuit breaker	Zuhal trips after 5 consecutive failures; half-open canary after 600s — prevents runaway spend

2. Component Breakdown

pipeline/producer.py

Reads input JSONL in chunk_size-record batches. Each chunk runs via asyncio.gather — fully parallel within the chunk.

Stage 1 — DNS probe (pipeline/utils/dns_probe.py)

Derives domain stems from the business name: 1. Lowercase 2. Strip legal suffixes (LLC, Inc, Corp, Foundation, Association) 3. Strip geographic terms (North Carolina, Raleigh) 4. Produce: joined (acmecorp), hyphenated (acme-corp), initials (ac for 3+ words) 5. Filter: minimum 5 chars, valid hostname characters only

For each stem, probes .com, .net, .org simultaneously. One shared aiodns.DNSResolver(timeout=3, tries=1) per producer — created once in **init**, passed to every call.

Stage 2 — Serper enrichment (pipeline/utils/serper_client.py)

Query construction:

Condition	Query
-----------	-------

without strategy, no domain	"Business Name" North Carolina contact email
without strategy, domain known	site:domain.com contact
with strategy, no domain	"First Last" "Business Name" North Carolina email contact
with strategy, domain known	"First Last" "Business Name" email contact site:domain.com

If site: query returns no emails, automatically retries without site: filter.

Email extraction filter: - known_domain present: keep only @known_domain and subdomain emails - known_domain absent: discard emails from FALLBACK_DOMAIN_BLOCKLIST (prevents aggregator false positives such as greatnonprofits.org) - with strategy: discard snippet emails where local part does not fuzzy-match agent name (score under 75)

Pattern generation (pipeline/utils/email_patterns.py)

```
without strategy -> info, contact, hello, office, admin (top 3)
with strategy    -> first.last, flast, firstlast, first, f.last, ... (top 5)
```

Templates reordered by pattern_stats success rate for the record's mx_provider. Unseen templates keep their default order.

pipeline/bbops.py

Standalone synchronous subprocess called by orchestrator/stage.py between producer and consumer.

1. Queries all records with record_state=DISCOVERED
2. Submits all candidate_emails in batches of 1,000 to POST /jobs/batch
3. Polls GET /batches/{batch_id} until status=done or 30-min timeout
4. For each record: picks best result (valid > catch_all > error > invalid)
5. Updates: record_state=VALIDATED, zuhal_status=bbops_valid (or VALIDATION_FAILED)

Records settled by bbops are fully resolved — consumer never touches them.

pipeline/consumer.py

Atomic claim prevents double-processing:

```
UPDATE records
  SET record_state = 'VALIDATING', updated_at = datetime('now')
WHERE id IN (SELECT id FROM records WHERE record_state = 'DISCOVERED' LIMIT 10)
RETURNING *
```

Per-record validation flow:

```
for email in candidate_emails:
    if cost_tracker.ceiling_reached():
        -> COST_SKIPPED, return

    if is_microsoft_mx(mx_provider):
        result = check_microsoft_email_async(email)
        if result == valid -> VALIDATED (ms_valid), return
        if result == invalid -> skip to next candidate
        # unknown/throttled/error -> fall through to Zuhal

    result = zuhal.validate(email)
    if result.verdict in (valid, accept-all):
        -> VALIDATED, record pattern_stats, return
    # invalid/unknown/disposable -> try next candidate

-> VALIDATION_FAILED (all candidates exhausted)
```

Stale recovery: recover_stale_validating() on startup resets rows stuck in VALIDATING for over 5 minutes back to DISCOVERED.

SQLite schema

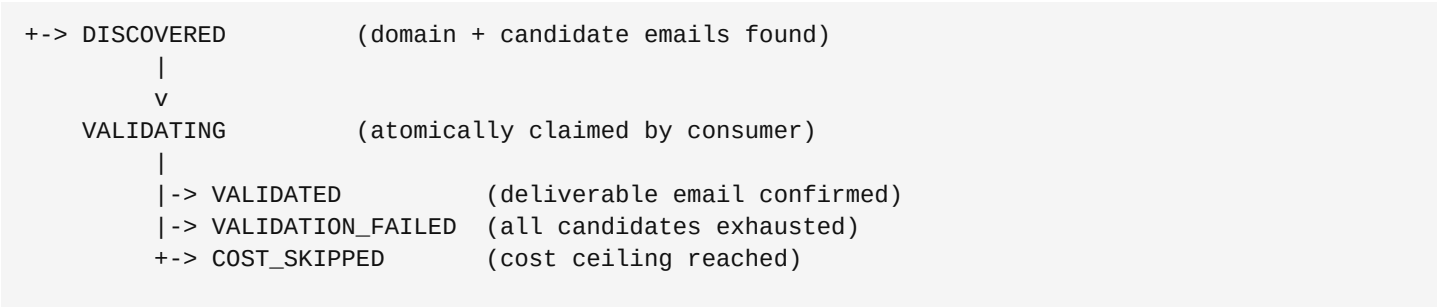
Tables: records, checkpoints, stats, failures, pattern_stats, enrichment_cache, verifier_jobs

WAL pragmas set in init_db():

Pragma	Value	Purpose
journal_mode	WAL	Concurrent reads during writes
synchronous	NORMAL	Performance without full fsync
busy_timeout	10000	Wait up to 10s on lock contention
cache_size	-64000	64 MB page cache
mmap_size	268435456	256 MB memory-mapped I/O
wal_autocheckpoint	1000	Checkpoint every 1000 pages
temp_store	MEMORY	Temp tables in RAM

3. Record State Machine

```
RAW
|
|-> DISCOVERY_FAILED    (DNS + Serper both failed)
|
|-> DISCOVERING          (transient error; retry eligible)
|
|      |
|      +--> (retried on next producer pass)
|
```



State	Terminal	Re-queueable
RAW	No	—
DISCOVERING	No	Yes (auto on next producer pass)
DISCOVERED	No	—
VALIDATING	No	Yes (via recover_stale_validating)
VALIDATED	Yes	No
VALIDATION_FAILED	Yes	Yes (via pipeline reset)
DISCOVERY_FAILED	Yes	Yes (via pipeline reset)
COST_SKIPPED	Yes	Yes (raise ceiling and re-run)

4. Output Schema

valid_emails.csv — 9 columns

Column	DB source	Values
unique_id	records.unique_id	{filing_id}__{agent_id}
business_name	records.business_name	Free text
agent_name	records.agent_name	Free text
state	records.state	NC, TX, etc.
email	records.candidate_email	Validated email address
zuhal_status	records.zuhal_status	See table below
confidence_tier	computed from zuhal_score	high / medium / low
discovery_method	records.discovery_source	dns / serper / input
validation_method	derived from zuhal_status	zuhal / ms_probe / bbops

zuhal_status vocabulary

Value	Source	Meaning
valid	Zuhal	Mailbox exists and accepts mail
accept-all	Zuhal	Catch-all domain; individual existence unverifiable
ms_valid	MS probe	Azure AD user confirmed to exist
bbops_valid	bbops.io	SMTP-level confirmation via batch verifier
invalid	Zuhal or MS	Address does not exist
unknown	Zuhal	SMTP server inconclusive
disposable	Zuhal	Temporary email provider detected

Confidence scoring

Additive score 0 to 4. Tier: 3 or above = high, 2 = medium, 1 or below = low.

Criterion	Points	Applies when
Email domain fuzzy-matches candidate domain (ratio ≥ 85)	+1	Always
Local part fuzzy-matches agent name variants (ratio ≥ 75)	+1	with strategy only
Local part is NOT a generic prefix	+1	with strategy only
Verdict is valid (not accept-all)	+1	with strategy
Local part IS a generic prefix	+1	without strategy
Verdict is valid (not accept-all)	+1	without strategy

Generic prefixes: info, contact, hello, admin, support, sales, help.

5. Cost Model

Per-call pricing

Service	Cost
---------	------

Serper	\$0.001 per call
Zuhal	\$0.0005 per call
MS probe	Free
bbops	Per contract

Cost per record scenario

Path	Cost
Discovery failed	\$0.001 (Serper only)
Discovered, MS probe validates	\$0.001 (Serper + free MS probe)
Discovered, 1 Zuhal pass	\$0.0015
Discovered, 3 Zuhal attempts (all fail)	\$0.0025

Scale projections

Records	Projected cost
10,000	~\$12
100,000	~\$120
300,000 (full NC dataset)	~\$350

6. Performance

Stage latency — 500 live NC records

Stage	Median	p95	Max
DNS probe	15,288 ms	26,179 ms	29,779 ms
Serper	1,122 ms	12,462 ms	159,128 ms
Zuhal	6,066 ms	13,377 ms	20,362 ms

DNS median of 15s is inherent to the dataset: 80% of NC nonprofits have no web domain, so each record exhausts all stem x TLD combinations before failing. Each failing combination caps at 3s (timeout=3, tries=1 on the shared resolver). Before the shared resolver fix, system-default DNS timeout was 10–30s per query.

Zuhal median of 6s includes a mandatory 0.5–2.5s randomized anti-fingerprint delay plus SMTP handshake latency.

Concurrency defaults

Parameter	Default	Safe ceiling
dns_concurrency	100	200
serper_concurrency	10	20
zuhal_concurrency	3	10
zuhal_rate_limit	200 per hour	Per Zuhal plan

7. Live Run — 500 Records

Date: 2026-04-30 Input: input/nc_retry_300k.jsonl (first 500 records) Mode: Live — real DNS, Serper, MS probe, and Zuhal calls

Summary

Metric	Value
Records processed	500
Discovery failed	401
Validation failed	55
Validated emails	44
Overall yield	8.8%

Confidence

Tier	Count
High	6
Medium	14
Low	24

Verdict

Verdict	Count
Valid	35
Accept-all	7
MS Valid	2

Discovery Method

Method	Count
DNS	15
Serper	1
Not recorded	28

Validation Method

Method	Count
Zuhal	42
MS Probe	2
bbops	0

Cost

Metric	Value
Total	\$0.5785
Per record	\$0.00116
Per validated email	\$0.0131
Serper calls	500
Zuhal calls	157
MS probe calls	~20 (free)